Network Deployment Guide (English) — v3.0

This document serves two audiences:

- 👨‍💼 **Owner / General users**: see diagrams, analogies, and "where is my data?"
- 👨‍💻 **Technical engineers / IT**: see configs, ports, nginx / VPN examples

v3.0 core promise: This is a **desktop conversational ERP for small manufacturers**. Employees open Chrome in the office, do all CRUD with one-sentence Chat. Main factory data stays on-premise, MESH multi-factory data does not leak.

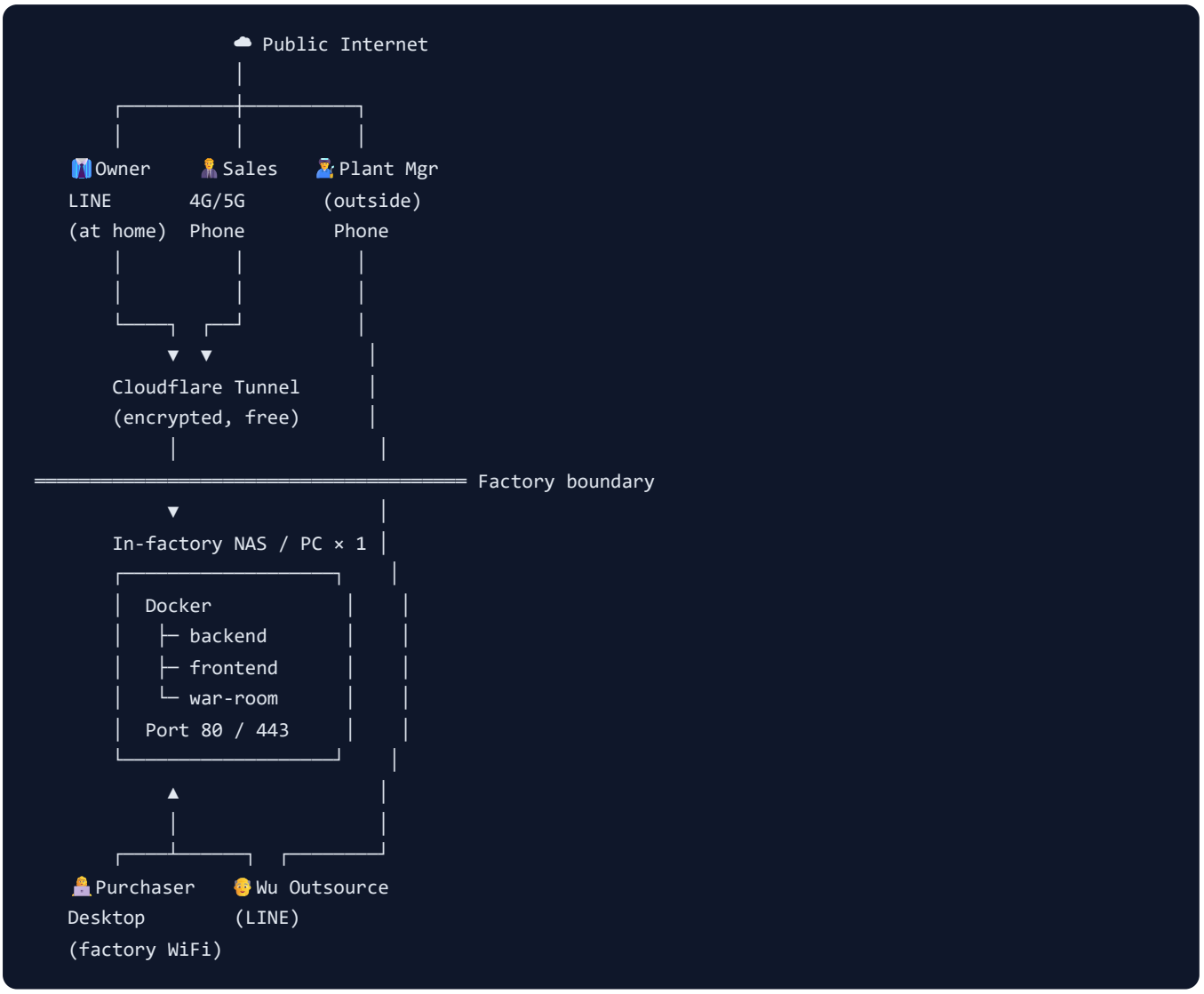
⚡ **v3.0 Strategic Pivot Notice (2026-05-15)** The "LINE Bot Webhook: Reaching You via LINE" chapter is **deprecated** in v3.0. "Outsource partners use LINE without registration" / "salespeople check from mobile" are v2 legacy.

🎯 30-Second Owner Summary

```
┌──────────────────────────────────────────────────┐
│  Where Does My Factory Data Go?                  │
├──────────────────────────────────────────────────┤
│                                                  │
│  1. Server runs on YOUR computer in YOUR factory │
│     → Data 100% never leaves the premise         │
│                                                  │
│  2. Staff use phone/PC via factory WiFi          │
│     → Fast, no Internet required                 │
│                                                  │
│  3. Salespeople use 4G/5G to reach factory       │
│     → Encrypted tunnel — nobody can eavesdrop    │
│                                                  │
│  4. Owner asks via LINE                          │
│     → LINE message → Cloudflare encrypted → factory│
│     → Reply goes back through same tunnel        │
│                                                  │
│  5. Outsource (e.g. plating shop) reports via LINE │
│     → No registration, no app to install         │
│     → Scan QR — main factory knows instantly     │
│                                                  │
│  ✅ Data NEVER uploaded to cloud                 │
│  ✅ Always encrypted                             │
│  ✅ Pull the network cable anytime to go offline │
│                                                  │
└──────────────────────────────────────────────────┘
```

📐 Three Typical Deployment Scenarios

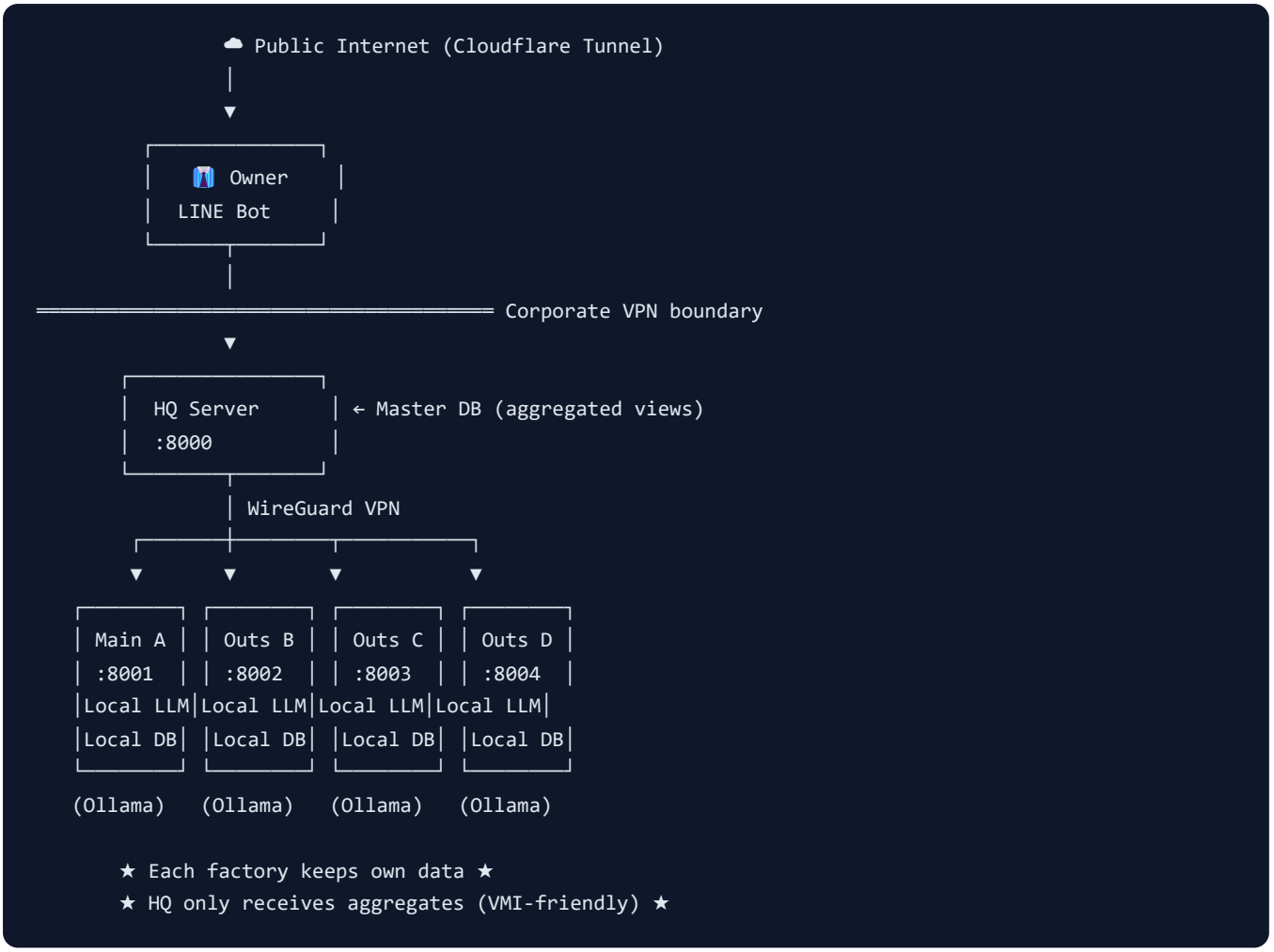
Scenario A: Single Factory (10-50 people) — Most Common



Features:

- ✅ Single machine (min i5 / 8GB RAM / 100GB SSD)
- ✅ Monthly cost: electricity + Cloudflare (free)
- ✅ Data 100% on-premise
- ⚠️ Factory power outage = service down (recommend UPS)

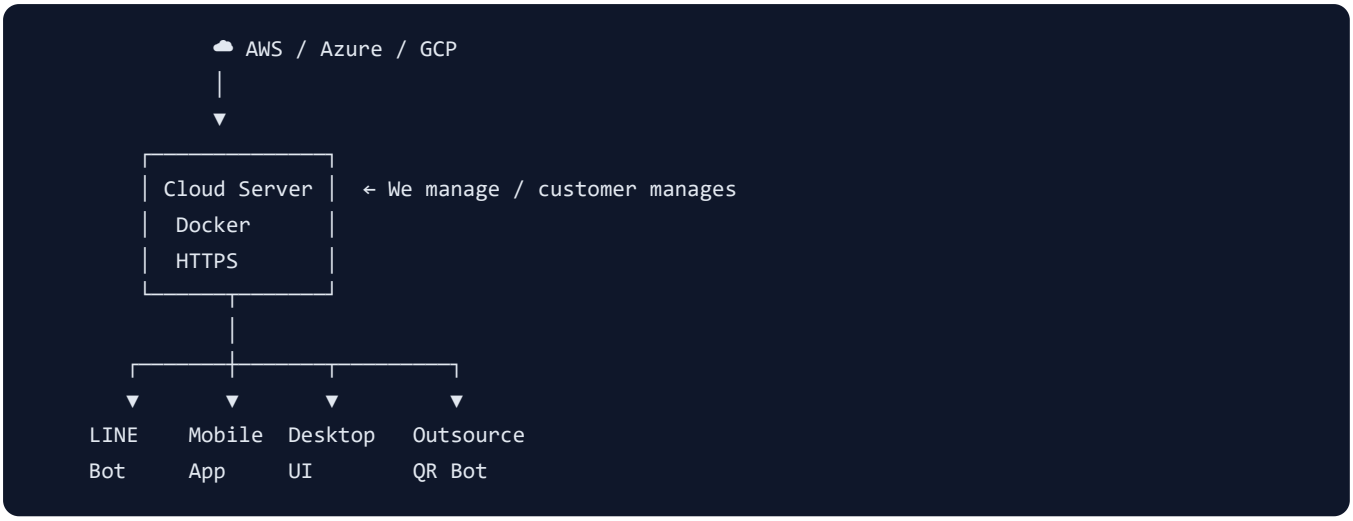
Scenario B: MESH Multi-Factory (HQ + Branches + Outsource)



Features:

- ✅ Per-factory data sovereignty (stays local)
- ✅ HQ unified view across chain
- ✅ Factory survives Internet outage (offline-first)
- ⚠️ VPN setup required (WireGuard is simple)

Scenario C: Cloud-Hosted (for small factory without IT)



Features:

- ✅ Zero maintenance
- ✅ Auto backup
- ✅ Scale on demand
- ⚠️ Data in cloud (trust provider)
- ⚠️ Higher subscription cost

🔌 Complete Port Reference

Port	Service	Public Exposure	Purpose
80	Frontend (nginx)	✅ Public / Internal	Desktop UI + API reverse proxy
443	Frontend (HTTPS)	✅ Public	SSL-encrypted version
8000	Backend (FastAPI)	❌ Internal only	REST API + SSE
8001-8003	Factory Nodes	❌ Inside VPN	MESH factory nodes
8080	War-Room	🟡 Internal / Mgmt	Real-time dashboard (display-only)
5432	PostgreSQL	❌ Backend only	Database
6379	Redis (Phase 5)	❌ Backend only	Cache / Queue
9000-9001	MinIO (Phase 2)	❌ Backend only	Image storage
11434	Ollama (Local LLM)	❌ Factory node only	Privacy LLM

Firewall rules (ufw example):

```
# Production firewall
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp    # SSH (better with source IP restriction)
sudo ufw allow 80/tcp    # HTTP (auto redirect to HTTPS)
sudo ufw allow 443/tcp   # HTTPS
sudo ufw enable
```

👷 For Engineers: Complete Reverse Proxy Config

Full nginx production config

```
upstream backend {
    server localhost:8000;
    keepalive 32;
}

# HTTP → HTTPS forced redirect
server {
    listen 80;
    server_name erp.your-company.com;
    return 301 https://$host$request_uri;
}

# HTTPS main service
server {
    listen 443 ssl http2;
    server_name erp.your-company.com;

    ssl_certificate     /etc/letsencrypt/live/erp.your-company.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/erp.your-company.com/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         HIGH:!aNULL:!MD5;

    # Security headers
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Frame-Options          "SAMEORIGIN" always;
    add_header X-Content-Type-Options   "nosniff" always;

    client_max_body_size 20m;

    # ── SSE-specific (must match before /api/) ──
    location = /api/events/stream {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header   Host $host;
        proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_buffering    off;
        proxy_cache        off;
        proxy_read_timeout 86400s;
        proxy_set_header   X-Accel-Buffering "no";
    }

    # ── General API ──
    location /api/ {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header   Host $host;
        proxy_set_header   X-Real-IP $remote_addr;
        proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
```

```
        proxy_connect_timeout 10s;
        proxy_read_timeout    90s;  # leave room for LLM

        # Rate limiting (anti brute-force)
        limit_req zone=api burst=20 nodelay;
    }

    # ── LINE Bot webhook (must be publicly reachable) ──
    location = /api/line/webhook {
        proxy_pass http://backend;
        proxy_set_header Host $host;
    }

    # ── SPA frontend ──
    location / {
        root /var/www/llm-erp;
        try_files $uri $uri/ /index.html;
    }
}

# Rate limit zone (put in http {} block)
# limit_req_zone $binary_remote_addr zone=api:10m rate=30r/s;
```

Let's Encrypt free SSL

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d erp.your-company.com
# Auto-issues + auto-renews
```

🌐 LINE Bot Webhook: Making LINE Reach You

Problem

LINE Bot requires **your server has a public URL**. Small factories usually don't have static IPs and don't want to open firewall.

Three solutions (recommended top-down)

Solution A: Cloudflare Tunnel (recommended ⭐⭐⭐⭐⭐)

Pros: Free, no firewall change, auto HTTPS, no traffic limit

```
# 1. Install cloudflared
curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64.deb -o cf.deb
sudo dpkg -i cf.deb

# 2. Login to Cloudflare
cloudflared tunnel login

# 3. Create tunnel
cloudflared tunnel create llm-erp

# 4. Configure DNS (add CNAME in Cloudflare DNS)
# erp.your-company.com → <tunnel-id>.cfargotunnel.com

# 5. Start tunnel
cloudflared tunnel route dns llm-erp erp.your-company.com
cloudflared tunnel --url http://localhost:8000 run llm-erp
```

Set up as systemd service for auto-start on boot.

Solution B: ngrok (for testing)

```
ngrok http 8000
# Get https://xxxxx.ngrok.io
# Paste in LINE Developer Console
```

> ⚠️ Free tier resets URL on restart. Use paid or Cloudflare for production.

Solution C: Static IP

Apply for static IP (~$50/month), open port 443. **Not recommended** — high maintenance burden.

🔐 MESH Multi-Factory VPN (WireGuard)

Why WireGuard

- **Simple**: Config 80% shorter than OpenVPN
- **Fast**: Kernel-level implementation
- **Secure**: Modern cryptography

HQ server config

```
# /etc/wireguard/wg0.conf (HQ)
[Interface]
PrivateKey = <HQ_PRIVATE_KEY>
Address = 10.0.0.1/24
ListenPort = 51820

[Peer]
# Factory A
PublicKey = <FACTORY_A_PUBLIC_KEY>
AllowedIPs = 10.0.0.2/32

[Peer]
# Factory B (outsource)
PublicKey = <FACTORY_B_PUBLIC_KEY>
AllowedIPs = 10.0.0.3/32
```

Factory side config

```
# /etc/wireguard/wg0.conf (Factory A)
[Interface]
PrivateKey = <FACTORY_A_PRIVATE_KEY>
Address = 10.0.0.2/24

[Peer]
PublicKey = <HQ_PUBLIC_KEY>
Endpoint = hq.your-company.com:51820
AllowedIPs = 10.0.0.0/24
PersistentKeepalive = 25
```

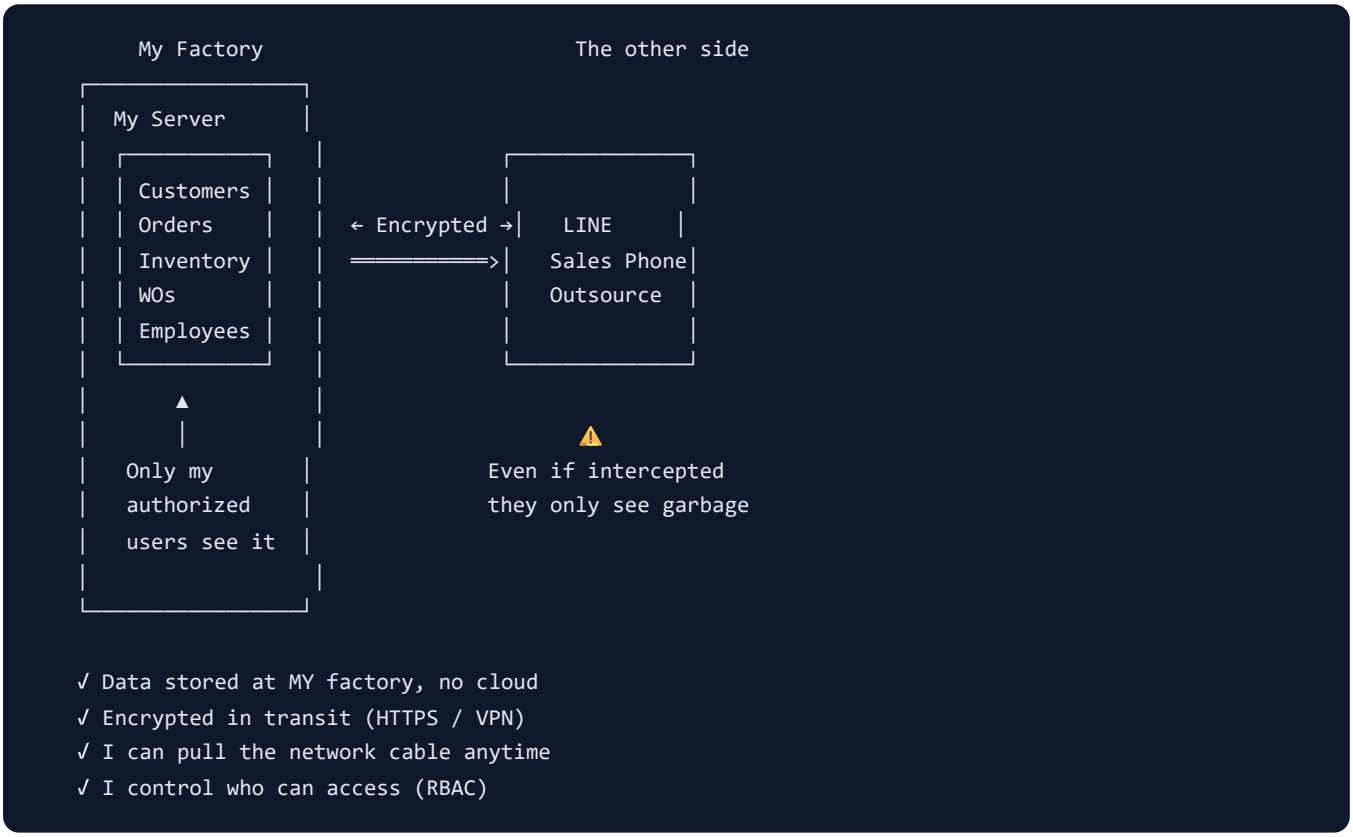
Start

```
sudo wg-quick up wg0
sudo systemctl enable wg-quick@wg0
```

Factory node docker-compose

```yaml
factory-a:
  command: ["python", "factory_node.py"]
  environment:
    FACTORY_ID: factory-a
    HQ_URL: http://10.0.0.1:8000   # Use VPN internal IP
    PORT: 8001
  network_mode: host  # Use host network for WireGuard
```

👴 "Where Is My Data?" Diagram for Owner



🛠️ Production Checklist (for your IT)

Must-do before production:

- [ ] `JWT_SECRET` changed to random 64-char (`openssl rand -hex 32`)
- [ ] `DEBUG=false`
- [ ] `LOG_JSON=true` (for ELK / Loki)
- [ ] `CORS_ORIGINS` set to real domain (no `*`)

- [ ] Demo `admin` account: strong password
- [ ] Firewall: only 80/443 (internal 5432)
- [ ] HTTPS configured (Let's Encrypt)
- [ ] Backup strategy: daily + weekly
- [ ] Monitoring / alerts (Phase 7)
- [ ] LINE Bot webhook URL configured

🚀 One-Click Start Script (for factory with no IT)

Save as `start.sh`:

```bash
#!/bin/bash
# LLM-ERP one-click start script

set -e

echo "🚀 Starting LLM-ERP..."

# 1. Check Docker
if ! command -v docker &> /dev/null; then
  echo "❌ Install Docker first: https://docs.docker.com/get-docker/"
  exit 1
fi

# 2. First-time setup
if [ ! -f backend/.env ]; then
  echo "📝 First run, creating config..."
  cp backend/.env.example backend/.env
  # Auto-generate JWT_SECRET
  SECRET=$(openssl rand -hex 32)
  sed -i.bak "s|change-me-in-production-please-use-openssl-rand-hex-32|$SECRET|" backend/.env
  rm -f backend/.env.bak
  echo "✅ JWT_SECRET auto-generated"
fi

# 3. Start
echo "🐬 Starting containers..."
docker compose up -d --build

# 4. Wait for backend
echo "⏳ Waiting for backend (up to 60s)..."
for i in {1..30}; do
  if curl -fsS http://localhost:8000/api/health > /dev/null 2>&1; then
    echo "✅ Backend ready"
    break
  fi
  sleep 2
done

# 5. First seed
if [ ! -f backend/.seeded ]; then
  echo "🌱 First run, loading demo data..."
  docker compose exec -T backend python -m scripts.seed
  touch backend/.seeded
fi

echo ""
echo "🎉 Done! Open in browser:"
echo ""
echo "   Desktop UI:   http://localhost:5173"
echo "   War Room:     http://localhost:8080"
echo "   API Docs:     http://localhost:8000/docs"
```

```
echo ""
echo "   Login: admin / admin123"
```

Run: `chmod +x start.sh && ./start.sh`

📞 Troubleshooting

Symptom	Likely Cause	Fix
Can't see Dashboard	backend not up yet	Wait 30s / check `docker compose logs backend`
LINE Bot no reply	Wrong webhook URL	Check Cloudflare Tunnel is running
Factory can't reach HQ	VPN not connected	`sudo wg show` to check peer status
Mobile can't connect	Cross-origin issue	Verify nginx proxy + CORS_ORIGINS
SSE events not arriving	nginx not flushing	Use our provided nginx.conf

---

📚 Further Reading

- ARCHITECTURE_DIAGRAM.md - System topology
- ADMIN_GUIDE.md - Admin guide
- DEPLOYMENT.md - Full deployment
- USER_MANUAL_EN.md - User manual

Last updated: 2026-05-14 **Chinese version**: NETWORK_DEPLOYMENT_ZH.md